

CSCE 637 COMPLEXITY THEORY · MODULE 10 · LECTURE NOTES

Interactive Proofs

What you can be convinced of by a conversation.

MODULE OUTCOMES

Define an interactive proof and explain the two conditions.

Explain the graph non-isomorphism protocol.

State $IP = PSPACE$ and explain arithmetisation.

State the PCP theorem and explain what it says.

Explain zero-knowledge and its practical use.

1 The model

AN INTERACTIVE PROOF IS A CONVERSATION WITH A RANDOMISED VERIFIER

A computationally unbounded prover and a polynomial-time randomised verifier exchange messages for polynomially many rounds. The verifier then accepts or rejects.

Completeness: if the statement is true, an honest prover convinces the verifier with high probability.

Soundness: if the statement is false, *no* prover — however powerful or dishonest — convinces the verifier except with small probability. **The quantifier over provers is the important part.**

And both ingredients are essential. Interaction alone adds nothing: if the verifier is deterministic, its questions are predictable, so the prover can simply send the entire transcript in advance — **and you are back to NP** (Module 03 §1's certificate). **It is the randomness, hidden from the prover, that the prover cannot anticipate**, and §1's protocol shows exactly how that is exploited.

TO PROVE: G_1 and G_2 are NOT isomorphic.
(No short certificate is known -- Module 05 section 1.)

THE PROTOCOL

the VERIFIER picks i in $\{1,2\}$ at random, and a random permutation; sends the permuted copy of G_i
the PROVER says which of G_1, G_2 it came from
repeat k times; accept if the prover is always right

IF THEY ARE NOT ISOMORPHIC: the prover (being unbounded) can always tell, because the permuted graph is isomorphic to exactly one of the two. Always right. ACCEPT.

IF THEY ARE ISOMORPHIC: the permuted graph is isomorphic to BOTH, so the prover has no information and is guessing. Right with probability 2^{-k} . REJECT, almost certainly.

NOTE WHAT MADE IT WORK: the verifier's random choice of i is HIDDEN FROM THE PROVER. A deterministic verifier's choice would be predictable, and the protocol would prove nothing at all.

SO INTERACTION PLUS RANDOMNESS CAN CERTIFY SOMETHING THAT HAS NO CERTIFICATE.

2 IP = PSPACE

IP = PSPACE, BY ARITHMETISATION

Interactive proofs capture exactly PSPACE — so **anything decidable in polynomial space can be proved to a polynomial-time verifier by conversation**, which is far more than NP and was a considerable surprise when proved in 1990.

The technique is arithmetisation: convert the quantified Boolean formula of TQBF (Module 05 §2) into a polynomial over a finite field, with AND becoming multiplication and the quantifiers becoming sums or products over the field. **The prover then makes claims about the polynomial's values, which the verifier spot-checks at randomly chosen points.**

And it works because low-degree polynomials are rigid: two distinct polynomials of low degree agree at only a few points, so they disagree almost everywhere — which means a prover who lies about the polynomial is caught by a random check with high probability. **The rigidity of polynomials is doing all the work**, and it is the same property behind error-correcting codes and Module 08's identity testing.

And this is the result that does not relativise (Module 07 §3) — **which is why the technique mattered methodologically**, far beyond the theorem itself, **and why the algebrisation barrier had to be constructed afterwards** to determine whether it could reach P versus NP. **It could not.**

3 The PCP theorem

PCP: EVERY NP PROOF CAN BE REWRITTEN SO THAT READING A CONSTANT NUMBER OF BITS SUFFICES

$\text{NP} = \text{PCP}(\log n, 1)$. Every statement in NP has a proof that a verifier using $O(\log n)$ random bits and reading a *constant* number of the proof's bits can check. Three bits suffice, in the sharpest versions.

A true statement's correctly written proof always passes; a false statement's purported proof fails with constant probability — from reading three bits of a proof that may be gigabytes long.

The mechanism is that the proof is written in a highly redundant, error-correcting form, so that any error is spread throughout the whole encoding rather than localised — and a random spot-check therefore finds it. The proof format, not the verifier's cleverness, is what makes it possible.

And its most useful consequence is hardness of approximation (Module 11, and CSCE 669 Module 10 §4's inapproximability results) — the gap between a passing and a failing proof becomes a gap in an optimisation problem's achievable value, and that gap is what makes inapproximability provable. Before PCP, essentially no inapproximability results were known.

Consequence	Why it follows
Hardness of approximation, as a field	The accept/reject gap becomes an approximation gap (Module 11 §1) — which is the whole technique.
MAX-3SAT cannot be approximated better than $7/8$	Directly from PCP, and $7/8$ is achievable by a simple randomised algorithm — so the bound is tight, which is an unusually satisfying situation.
Set cover's logarithmic ratio is optimal	Which CSCE 669 Module 10 §4 asserted, and this is the result that proves it.
Succinct, spot-checkable proofs in practice	SNARKs and verifiable computation systems descend from PCP's ideas, through a long line of refinements.

The practical descendants are real: succinct proof systems deployed in cryptographic and blockchain systems are PCP's engineering grandchildren — the line runs through interactive oracle proofs and polynomial commitments rather than directly, but the core idea that a long computation can be verified by checking a few random positions of an encoded transcript is PCP's. Which is unusual for a result this abstract, and is worth knowing when the theorem seems remote.

4 Zero knowledge

ZERO-KNOWLEDGE: THE VERIFIER LEARNS THE STATEMENT IS TRUE AND NOTHING ELSE

A proof is zero-knowledge if the verifier's entire view of the conversation could have been *simulated* by an efficient algorithm with no access to the prover — so the transcript demonstrably carries no information beyond the statement's truth, since it could have been produced without anyone knowing a proof. The simulation definition is the subtle and correct one.

The classic example is graph three-colouring: the prover commits to a randomly permuted valid colouring, the verifier picks one edge and the prover opens just that edge's two commitments, revealing two different colours. Repeat many times. Each round reveals two differing colours — which the verifier could have generated itself — and nothing about the colouring.

And every NP statement has a zero-knowledge proof, assuming bit commitments exist — which follows from one-way functions. A striking general result: anything with a certificate can be proved without revealing the certificate.

The practical use is authentication and privacy-preserving verification — proving you hold an authorised credential without revealing which one, proving a transaction is valid without revealing its amounts, proving a computation was performed correctly without re-running it (CSCE 711's subject). This is the most directly applied result in the course.

- Interaction plus randomness certifies statements with no certificate — and graph non-isomorphism is the demonstration (§1).
- And the randomness must be hidden from the prover, which is precisely where the power comes from and why interaction alone is worthless.
- $IP = PSPACE$, by arithmetisation — and that is the technique that evaded the relativisation barrier (Module 07 §3), which is its larger significance.
- PCP rewrites NP proofs so that three bits suffice, which gives hardness of approximation (Module 11) and, eventually, deployed succinct proof systems.
- And every NP statement has a zero-knowledge proof, which is the most directly applied result in the course. This is the module where complexity theory produced deployed technology, which is worth noting in a course otherwise about what cannot be done.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Arora & Barak — chapters 8 and 11 (free draft)	Interactive proofs, $IP = PSPACE$, and the PCP theorem — the reference for this module.
Shamir — $IP = PSPACE$ (free)	§2's theorem, with the arithmetisation worked through.

Resource	What it covers for this module
Arora, Lund, Motwani, Sudan & Szegedy — Proof Verification and the Hardness of Approximation Problems (free)	§3's theorem , and the paper that created the hardness-of-approximation field.
Goldwasser, Micali & Rackoff — The Knowledge Complexity of Interactive Proof Systems (free)	§1 and §4's originating paper — the model and zero knowledge, in one place.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement the graph non-isomorphism protocol** and run it on isomorphic and non-isomorphic pairs.
- 2 **Measure the acceptance rate** and confirm the 2^{-n} bound.
- 3 **Make the verifier deterministic** and show the protocol proves nothing.
- 4 **Arithmetise a small Boolean formula** into a polynomial and verify the correspondence.
- 5 **Demonstrate polynomial rigidity** by comparing two low-degree polynomials at random points.
- 6 **Explain the PCP theorem's statement precisely**, including what the two parameters mean.
- 7 **Explain why an error-correcting proof format makes spot-checking work**.
- 8 **Implement the three-colouring zero-knowledge protocol** with a hash-based commitment.
- 9 **Write the simulator** and confirm it produces indistinguishable transcripts.
- 10 **Find a deployed system using zero-knowledge proofs** and describe what it proves and hides.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define an interactive proof and state completeness and soundness.
- 2 Why does interaction alone add nothing?
- 3 Give the graph non-isomorphism protocol and explain both cases.
- 4 State $IP = PSPACE$ and describe arithmetisation.
- 5 Why does polynomial rigidity make the protocol sound?
- 6 Why does this result matter methodologically?
- 7 State the PCP theorem and explain the mechanism.
- 8 Name four consequences of PCP.

- 9 Define zero-knowledge via simulation and give the three-colouring protocol.
- 10 What is the most directly applied result here, and where is it used?